Cybersecurity

Secured Application Development

An Introduction

Dr. Abdullmalek Alqobaty

Associated professor

Computer Engineering

Secured Application Development

Introduction

An Introduction

Introduction to Secured Applications

Look at the following three examples:

- **In 2009, hackers obtained user *credentials of over 30 million users* of mobile game publisher RockYou. They exploited a SQL injection to obtain the site's user table that uses unencrypted passwords.**
- **In 2010, a developer released a *Firefox extension called Firesheep that enabled eavesdroppers to obtain session IDs of Facebook and other sites logged in through the same Wi-Fi network.***
- **In 2017, hackers obtained the *records of over 130 million individuals from credit bureau Equifax.* The vulnerability was in a web framework Equifax was using, Apache Struts.**

These examples pointed that even large, reputable organizations can get hacked. In each instance, the breach was preventable.

In this course, we will look at the vulnerabilities that lead to these compromises, along with many others, and, of course, how to fix them.

Introduction to Secured Applications

Web application security is an ongoing war between hackers and developers.

If developers, simply learn techniques to *make applications* more secure, *without understanding the attacks, hackers will find other vulnerabilities to exploit*. Therefore,

- ***We must understand how hackers discover and exploit vulnerabilities.***
- ***We must also understand why defenses work and what attacks they do and do not prevent.***
- ***Most importantly, we must use several techniques together and choose a set of defenses for the requirements of our application***
- ***At the same time making our applications secure enough to prevent or minimize attack.***

Importance

The secure web application development is important for:

- ***Software engineers who develop, or want to develop, web applications and APIs.***
- ***Penetration testers who want to understand more about how web applications are written and how they are attacked.***
- ***Managers and security policymakers who want to know what threats web applications face and what measures can prevent attacks.***

Types of Attack

- **Web applications are applications whose UI is in a web browser that communicates with a server over *HTTP or HTTPS.***
  - **We also include *Websockets* and *HTTP streaming in this definition*, when the connection is made from a page delivered over HTTP or HTTPS.**
  - ***However, it also refers to APIs that communicate over HTTP and HTTPS*, even if they do not have a UI.**
- ***There are two broad categories for web application attacks:***
  - ***server side***
  - ***client side.***

Server–Side Attacks

Server-side attacks target code or services running on the server such as:

1. **Exploiting vulnerabilities in your code, such as SQL injection to perform malicious operations on your database.**
   - ***SQL injection vulnerabilities occur when user input form input fields, is concatenated with SQL query code.***
   - ***If the concatenation is not done safely,*** attackers can execute their own code on the database.
2. **Opening backdoors to obtain shell access on your server.**
   - ***Backdoors enable hackers to obtain shell access on your servers.*** Hackers are able to open them when there are certain vulnerabilities in your code or third-party servers, libraries, and frameworks.
   - ***An example of a vulnerability is code that executes a shell command****, concatenating text entered by the user in a form field with commands in the back-end code.* This may enable the hacker to run a shell command to open a port they can connect to. This is also discussed later in this course.

Server–Side Attacks

3. ***Exploiting permission misconfiguration to access files or URLs that should be unavailable***
   - ***Permission misconfiguration can include files having the wrong operating system permission*** *(writable by the web server user, for example)*, or
   - ***a misconfigured application permission system that allows unauthorized users access to URLs*** that should be password protected, or for admins only.

We look at these vulnerabilities in lecture 5 to 10.

Client–Side Attacks

Client-side attacks target the user and the user's browser. These include:

1. ***Cross-Site Request Forgery, exploiting the fact that a user is already logged into a service***
   - ***Cross-Site Request Forgery is an attack where a victim is enticed to follow a link on a site that they are logged into, but that performs an action in the attacker's favor.***
   - ***An example is a link to do a transfer from the victim's bank account to the attacker.*** If the victim is logged into the bank and the site is not properly defended against this attack, then the transfer may get executed, possibly without the victim being aware.
2. ***Cross-site scripting attacks aim to get JavaScript code executed by a victim through their browser.*** **This may be by uploading malicious code to an application or by sending a user a link with malicious JavaScript in an email.** ***They often target session IDs users have stored in their browser.***

▣ ***Defenses against client-side attacks aim to protect users when they unknowingly follow a malicious link.*** **These defenses do not necessarily protect your services from an attacker accessing them directly.** ***For this, we use server-side defenses.***

Client–Side Attacks

3. ***Clickjacking*, *getting a user to unknowingly follow a disguised link Cross-site scripting, where an attacker gets JavaScript code executed in the victim's browser to access their account or cookies***
   - ***Clickjacking attacks trick users into clicking on a malicious link.*** **The link may be disguised, for example, by superimposing it over a legitimate link and making it invisible.**

Defense in Depth

- **Defense in Depth is the practice of layering defenses.**
  - **For example, our code should be written in such a way as to prevent unauthorized users from accessing password file.**
  - ***We will ensure there are no SQL injection vulnerabilities and that unauthorized users cannot log into the server.***
  - ***However, in case these defenses fail****, we also make sure passwords are hashed so that if the table is compromised*, the hacker still does not obtain the passwords, at least not without a significant amount of work to crack them.
  - ***We mentioned RockYou at the start of this lecture. If users' passwords in their table had been hashed and they had better password policies (e.g., requiring digits and punctuation characters), hackers would not have obtained access to users' accounts, even with the SQL injection vulnerability.***

Defense in Depth

- **Defenses fall into three categories:**
  - ***Physical:*** ***Physical defenses include locks on doors, security cameras, etc.***
  - ***Technical:*** ***Technical defenses are the ones we code into our systems, such as pass-word authentication, checking form input, and cookie management.***
  - ***Administrative:*** ***Administrative defenses are human procedures such as training staff, four-eyes principles *, and monitoring logs. We usually use all of these categories to protect our applications.***

* The Four Eyes Principle (also two-person rule) is a widely used Internal Control mechanism that requires that any activity by an individual within the organization that involves Material Risk profile must be controlled (reviewed, double checked) by a second individual that is independent and competent.

Dr. Abdullmalek Alqobaty

Associated professor

Computer Engineering
Taiz University

Secured Application Development

Threat Modeling

Threat Modeling

Threat Modelling

In this lecture, we look at the fundamental task of understanding

- ***what we are trying to protect*** and
- ***where threats come from***.
- **There are a number of ways to achieve this.**
  1. ***The first method we will look at is asset modelling, which seeks to enumerate what we want to protect by looking at what is valuable to our organization.***
  2. **We will also look at the *STRIDE model*, a common *methodology for characterizing threats*.**
  3. **Then diving into a *data-flow-oriented approach to modelling the threat landscape*.**
- ***The asset modelling and STRIDE will enable us to understand***
  - ***the attack surface* and *attack vectors*.**
  - ***The entry points and attack processes hackers may use to gain access to our assets.***

Threat Modelling Definition

- ***The goal of threat modelling is to enumerate your system's threats,***
  - ***determine who they come from***, and ***plan how to address them.***

- **Defining some terms:**

- **An *asset* *is something we wish to protect.* It may be *tangible*, such as user data, or *less tangible*, such as your organization's reputation.**
- **A *threat* is what we wish to protect an asset from. Examples include disclosure of user data or defacement of the website.**
- ***A threat actor is an individual or organization interested in our assets.* Often, we think of the threat actor as a hacker, but not all threats are a result of hacking, for example, user error.**
- ***A vulnerability is a law in a system that can be exploited by threats.* An example is SQL injection in one of your web forms.**
- ***Impact is the outcome of a vulnerability being exploited.* The impact of a SQL injection vulnerability on a login page may result in financial loss for customers.**
- ***Risk is the potential for loss from a threat.* It encompasses the probability of exploitation and magnitude of the impact.**

I- Asset-Based Threat Modelling

- **The advantage of the asset-based approach is that it can be begun before development of the application starts.**
  - ***It is helpful in enabling developers to understand what must be protected, and how strongly, before they start designing the application or writing code.***
- **Assets, threats, and threat actors can be enumerated even if no vulnerabilities exist.**
  - ***This exercise, when performed before coding starts, can highlight some surprising threats and enable developers to address them at the design phase.***

Assets

Let us take an example to list the assets for an application.

- ***Consider a scientific group at a university who makes a climate change model available on the Internet. Users can enter environmental parameters, and the site then makes a climate prediction.***

  For maximum reach, the group decides not to require users to log in. **As the data on their site are public domain,** ***they do not initially believe the data must be secured.***

We may consider the following to be the assets:

- *Climate data*
- *The scientific model (M of VMC)*
- *Server availability*
- *Group's reputation*

Threats

- ***We next look at the threats to the assets.*** ***An example is shown in Table below.***

Asset	Threat
Climate data	Changing stored data
The scientific model	Changing code
Server availability	Denial of service on web server Heavy use of back-end servers
Group's reputation	Uploading malware to web server Defacing web pages

- ***From this analysis, it is clear that although the data are public and perhaps not worth preventing unauthorized users from reading,*** ***there is a threat of the data being written to, with potentially damaging consequences.***

Threat Actors

- ***We can use the assets and threats to identify threat actors:*** ***individuals or organizations interested in that asset and posing that threat.***
- ***Threat actors are often divided into categories:***
  - **Nation-states and their agents:** **Governments, and those acting on their behalf, may have a number of motivations.** ***They may wish to gain intelligence on other governments or foreign companies. They may wish to spy on their own citizens.***
  - **Organized crime:** **Criminal groups *after financial gain* from compromising systems.**
  - **Terrorist groups:** **Terrorists may be *interested in financial gain* but also in disrupting vital services and creating fear.**
  - **Hacktivists:** **Groups or individuals who are interested in furthering a cause.**
  - **Inside agents:** **People from within the organization who may be acting out of *revenge* or *financial gain*.**
  - **Script kiddies:** ***People who lack technical sophistication but know how to run scripts*.** **They often act to gain prestige among their peers.**
  - **Human error:** **Accidental compromise of a system due to human error. This may be developers or users.**

Threat Modelling

It is useful to list the actors interested in exploiting each threat as threat actors have varying degrees of sophistication and levels of interest.

- ***We can put the threats and threat actors in a table along with the sophistication of the actor and the impact of the threat.***
- ***In each cell of the table, that is, threat–threat actor combination, we put the interest the actor has in exploiting that threat.*** For our climate application, it might look like Table below.
- ***For example, the group may consider climate change activists or climate change deniers to have an interest in falsifying the data in order to further their cause, and the group may regard this interest as being high.***

Threat Modelling

- It is useful to list the actors interested in exploiting each threat as threat actors have varying and levels of interest.

Thread ▷ / ▽ Actor	Level of Interest	Changing Stored data	Changing code	Denial of Service on web server	Heavy use of back-end servers	Uploading malware	Defacing Web pages
Impact							
Nation-state		Low	Low	Low	Low	Low	Low
Organized crime		Low	Low	Low	High	Medium	Low
Terrorists		High	High	High	Low	Low	Low
Inside agents		Medium	Medium	Low	Low	Low	Low
Script kiddies		Low	Low	Low	Low	Low	Medium
Human error		Medium	Low	Medium	Low	Low	Low

Threat Modelling

Thread ▷ ▽ Actor	Level of Interest	Changing Stored data	Changing code	Denial of Service on web server	Heavy use of back-end servers	Uploading malware	Defacing Web pages
Impact		***High***	***High***	***Low***	***High***	***High***	***Medium***
Nation-state	**High**	**Low**	**Low**	**Low**	**Low**	**Low**	**Low**
Organized crime	***Medium***	**Low**	**Low**	**Low**	**High**	**Medium**	**Low**
Terrorists	***Medium***	**High**	**High**	**High**	**Low**	**Low**	**Low**
Inside agents	***Medium***	**Medium**	**Medium**	**Low**	**Low**	**Low**	**Low**
Script kiddies	***Low***	**Low**	**Low**	**Low**	**Low**	**Low**	**Medium**
Human error	***Medium***	**Medium**	**Low**	**Medium**	**Low**	**Low**	**Low**

Threat Modelling

- ***Based on this analysis, the developers can decide where to focus their defensive efforts.***
  - **There is, for example, little impact and likelihood of a denial-of-service attack on the web server,** but there is a **high interest**, and ***medium level of interest in*** use of their **back-end servers**.
  - ***The developers may therefore choose to rely on upstream network providers to filter out denial-of-service attacks but may take extra precautions to protect their back-end servers, for example, by throttling and logging requests.***

2- STRIDE Method

STRIDE is a method of categorizing risks. It stands for

- ***Spoofing***
- ***Tampering***
- ***Repudiation***
- ***Information Disclosure***
- ***Denial of Service***
- ***Elevation of Privilege***

▣ **It was proposed by Microsoft employees as an internal publication but has since been widely adopted.**

STRIDE Method

- ***Spoofing:*** *Spoofing is a type of cybercriminal activity where someone or something forges the sender's information and pretends to be a legitimate source, business, colleague, or other trusted contact for the purpose of gaining access to personal information, acquiring money, spreading malware, or stealing data.*
- ***Tampering:*** *The action of making changes to something that you should not such as data, usually when you are trying to damage it or do something illegal: Any tampering with the system risks making it less effective.*
- ***Repudiation:*** *A repudiation can be used to change the authoring information of actions executed by a malicious user in order to log wrong data to log files. Its usage can be extended to general data manipulation in the name of others. If this attack takes place, the data stored on log files can be considered invalid or misleading.*

STRIDE Method

- ***Information Disclosure:*** *Disclosure is the act of making confidential or sensitive information public without the consent of the owner. This can be intentional or unintentional and can lead to identity theft, fraud, or even reputational damage.*
- ***Denial of Service:*** *A DoS attack is a malicious attempt to overwhelm a web property with traffic in order to disrupt its normal operations.*
- ***Elevation of Privilege:*** *An elevation-of-privilege occurs when an application gains rights or privileges that should not be available to them. Many of the elevation-of-privilege exploits are similar to exploits for other threats.*

STRIDE Method

- ***Spoofing*** ***involves a threat actor using another person's or system's identity to fraudulently gain access.***
  - **An example** is using another user's session ID to log into an application.
  - **Another example** is a man-in-the-middle attacker pretending to be the server a user intended to access.
- ***Tampering*** ***involves changing data, code, or configuration.***
  - **An example** is an attacker resetting a different user's password. Another is uploading malware to a website.
- ***Repudiation*** ***is a user or attacker being able to deny performing an action.***
  - A user may deny receiving an order, or an attacker may delete server logs to hide their activities.

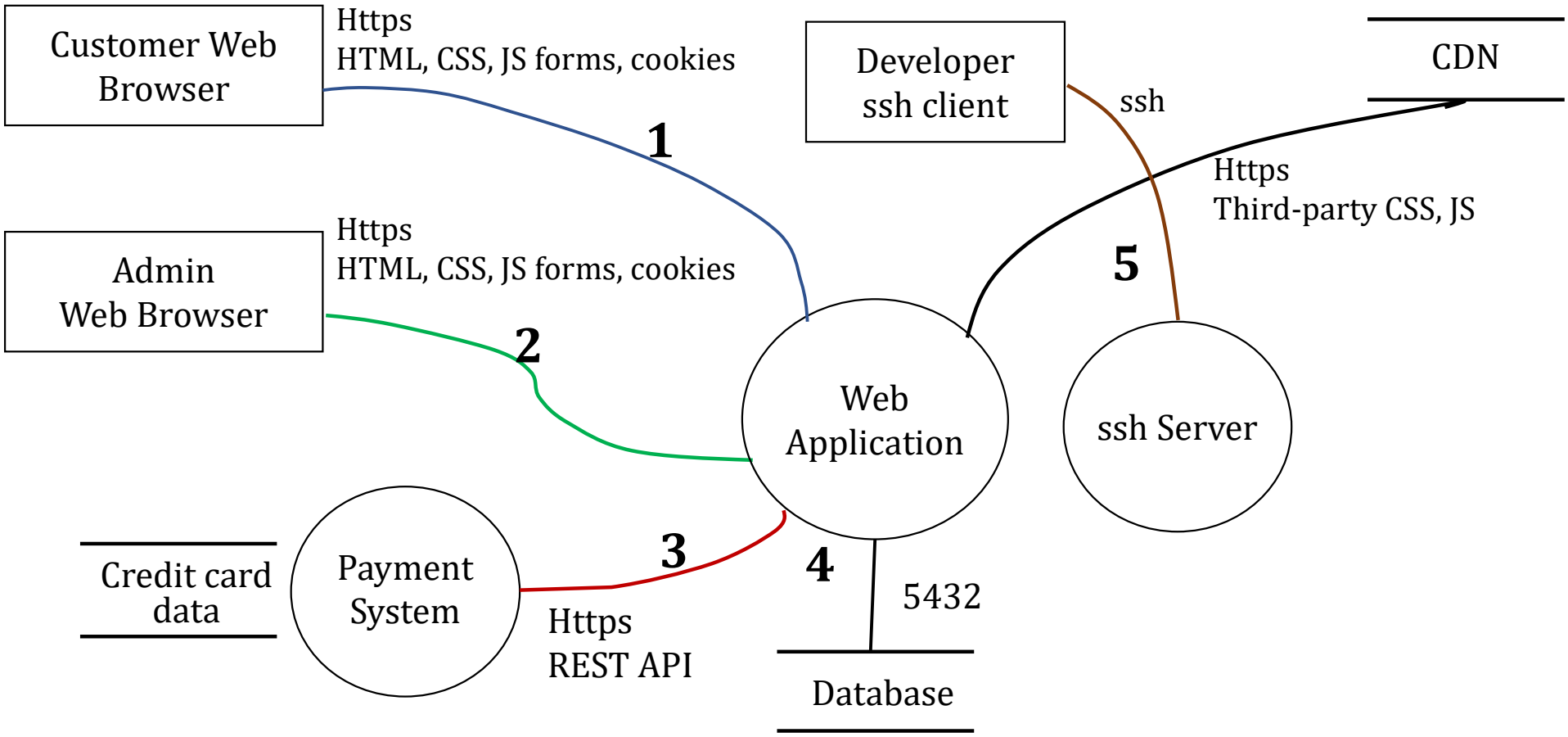
STRIDE Method

- ***Information Disclosure* is people gaining the ability to read data they are not entitled to.**
  - An example is obtaining the password list of registered users.
- ***Denial of Service* is making an application, server, or network unavailable to legitimate users**.
  - A well-known example is distributed *denial-of-service attack (DDoS)*, where an attacker orchestrates many servers to simultaneously connect to a target system, making it unresponsive to its intended users.
- ***Elevation of Privilege* is a threat actor gaining rights they are not entitled to,**
  - for example, administrator access to a web application.

3– Data–Flow Threat Modelling

- ***Data-flow diagrams*** ***are a popular engineering tool for designing how components of a system interact.*** They can also be useful for identifying threats.
- Data-flow diagrams describe systems in terms of processes, actors, data stores, and data flows.
  - **Processes are represented by circles.**
  - **Actors are represented by rectangles.**
  - **Data stores are represented by a pair of horizontal lines.**
  - **Data flows are presented by arcs.**

Data-Flow Threat Modelling



Data-Flow Threat Modelling

1. ***Customers connect to the application with a web browser over HTTPS****. The server sends HTML, CSS, and JavaScript files as response data.*
2. ***The client makes requests and sends form data. The client and server also exchange cookies.*** The same is true for administrative staff.
3. ***A connection is made to external payment server over a REST API.***
4. ***Application stores data in a SQL database, to which it connects over port 5432*** and also loads third-party code such as Bootstrap from an external ***CDN***.
5. **developers connect to the production VM over SSH as the production user.**

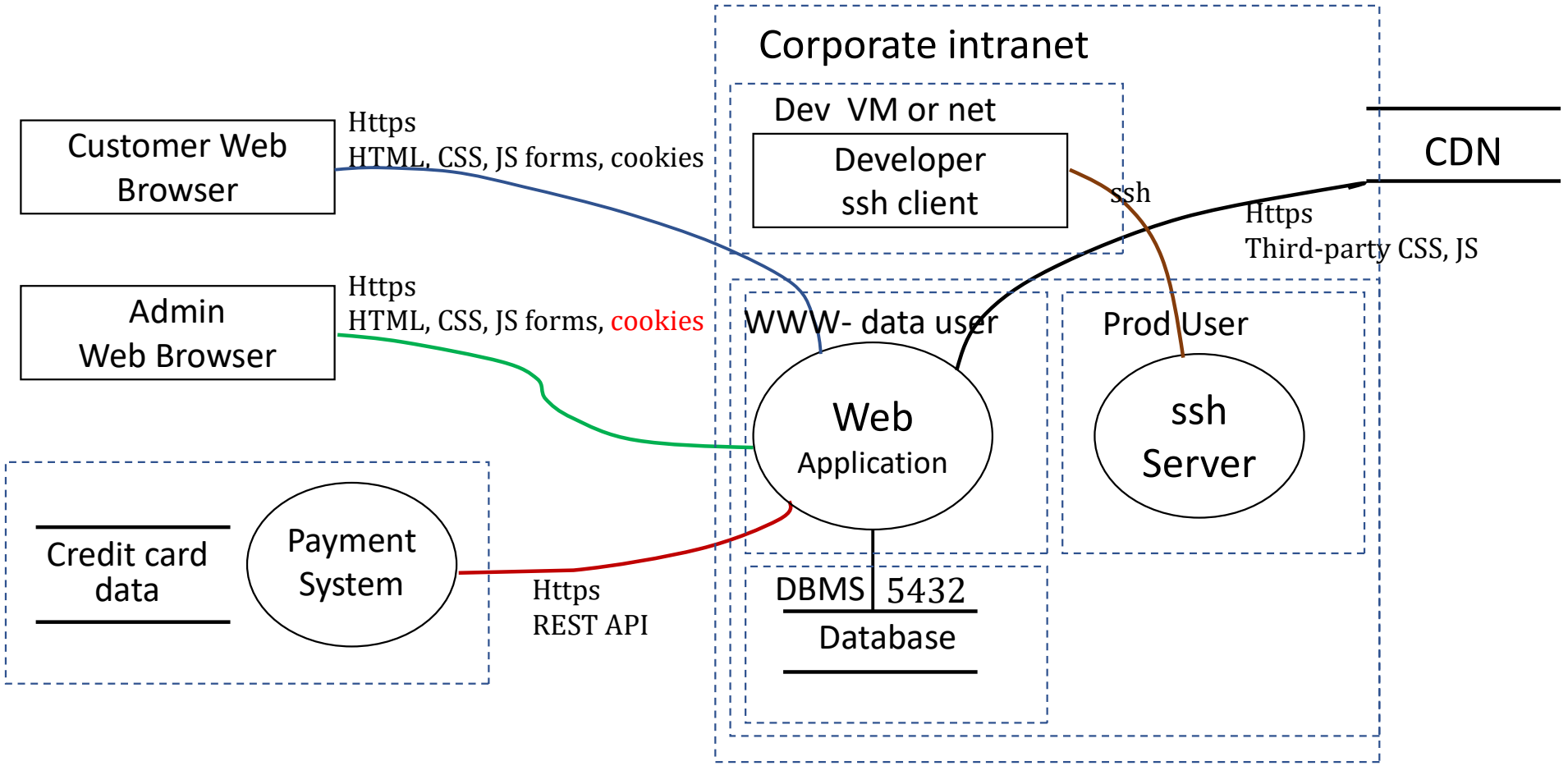
Trust Boundaries

Threats become clearer with ***adding*** ***trust boundaries*** to the diagram. ***Trust boundaries mark places where the level of trust changes.***

- ***There is a trust boundary between a corporate intranet and the public Internet*** - *only employees of the company are allowed access to the corporate Internet.*
- ***External users are restricted to accessing the application over HTTPS.***
- ***Trust boundaries can also exist between hosts.***
- ***There can also be trust boundaries between processes.*** For example, our web server runs as user www-data.
- **The database runs as user postgres.** ***The postgres user cannot edit files owned by www-data and vice versa.***

Trust Boundaries

The figure below shows the same data-flow model with ***trust boundaries***.



Trust Boundaries

The trust boundaries make further threats clear.

- For example, ***there is an escalation of privilege threat from the www-data user to the prod user.*** These threats may not exist as actual vulnerabilities, but it is useful to enumerate them anyway.
- They form a checklist when testing the implementation

Responding to Threats

Once a list of threats has been identified, our response can be classified into three categories:

- ***Accept*:** Accepting a threat means acknowledging it exists but either the likelihood of exploitation or the magnitude of the impact is small enough to be tolerated.
- ***Mitigate*:** Mitigating a threat means taking measures to reduce the chance of exploitation.
- ***Avoid*:** Avoiding a risk means changing the system to eliminate the threat, often by removing features.

Responding to Threats

Example

After a user logs in, a session cookie is stored. *If it is present in a request and is valid, the user does not have to enter a password.*

There is a threat of an attacker stealing a user's session cookie and spoofing its owner.

- **We can accept the risk**, considering it to be the responsibility of the user to protect their cookies.
- **We could mitigate the risk** by requiring the user to log in again if they connect from a new IP address. This would add inconvenience for the user but add security.
- **Avoiding the risk** would be to not use session cookies at all and require the user to log in with each request.

Attack Vectors

- ***Attacks on systems can be multistep and complex.*** ***A vulnerability might not seem serious until viewed as a step in a wider process. We call the whole process an attack vector***.

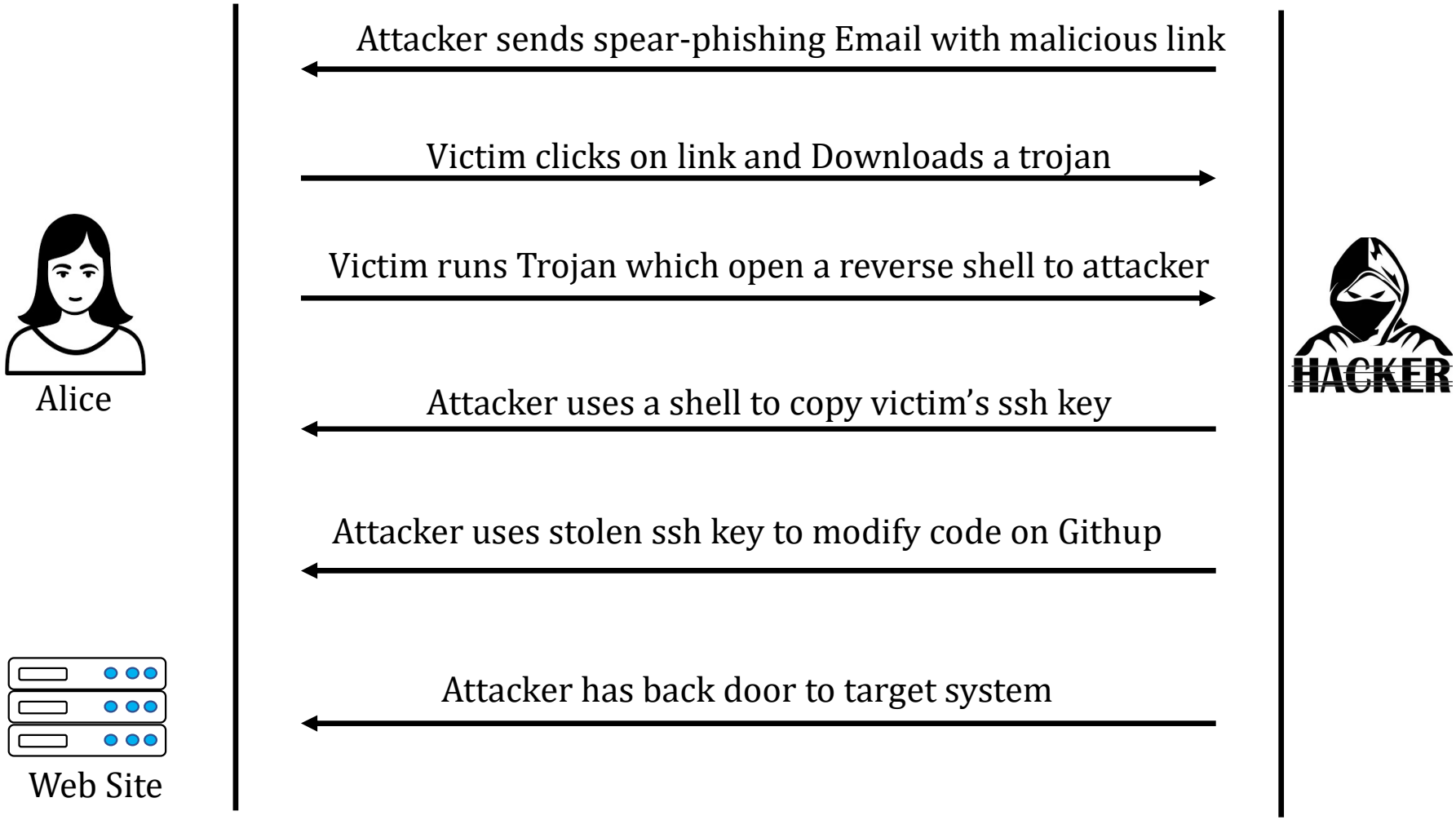
- ***The example below clear the concept.***
  See also the example diagram.

Attack Vectors: Example

- ***The attacker sends an individual a spear-phishing email with a link to malicious software.*** ***Spear-phishing emails are designed to convince the recipient to disclose information or click on a link.*** Unlike regular phishing emails, they are crafted specifically for an individual or group of people. ***In this example,*** ***it may appear to come from the victim's manager, asking them to install a particular application.***
- ***The victim clicks on the link and downloads the software which, is a Trojan*** (A malicious software disguised as something else).
- ***Once the reverse shell is established, the attacker uses it to download the victim's SSH keys.*** Among these keys is one that grants the attacker access to a GitHub repository.
- ***The attacker changes the source code in this repository,*** adding a backdoor for them to gain administrator access to the software.

In this example, the intended target was not the initial victim but a software system.

Attack Vectors



Attack Surfaces

The National Institute of Standards and Technology (NIST), defines the attack surface as:
The set of points on the boundary of a system, a system element, or an environment where an attacker can try to enter, cause an effect on, or extract data from that system, system element, or environment.

These can be physical or digital. We focus only on digital.

The attack surface consists of

- ***Places where data passes in and out of an application***
- ***Commands that can be executed on the application or its servers***
- ***Data used by or collected by the system***
- ***Code that protects the aforementioned or makes it available***

Attack Surfaces

- ***To map the attack surface, identify the following components in your application.***
  - ***URL endpoints***
  - ***HTTP headers and cookies***
  - ***Files***
  - ***Databases***
  - ***IP addresses and open ports***
  - ***Email accounts***
  - ***Form input pages and API endpoints***
  - ***Any other place where data can be submitted*** *(e.g., GET parameters) Interfaces to other systems*

Attack Surfaces

- ***Next, list user accounts and groups on the servers that own files or run services. This includes the OS accounts, database accounts, and application accounts*** *(e.g., admin accounts).*

  This means the attack surface for the database includes:

  - ***all URL endpoints that contain code that accesses the database.*** Particular attention should be paid to the set of those endpoints where SQL code is joined with user input.
  - ***The attack surface also includes any files that contain the database password, the database port 5432, and any host from which it is reachable.***
  - ***As the DBMS user can access the database without a password,*** *the attack surface includes any user account with access to the database.*

Attack Tree

- **An attack tree is a graphical representation of the various steps an attacker might take to exploit vulnerabilities and achieve specific malicious goals.**
- **It provides a visual and organized way to model the attack paths, potential vulnerabilities, and their dependencies. Attack trees are built using a hierarchical structure and can be subdivided into smaller attack trees to represent different attack vectors.**
- **How attack trees work**
  - **Root Node**: The top-most node represents the attacker's main goal.
  - **Branches**: Branches represent different methods or paths to achieve the goal or sub-goal.
  - **Leaf Nodes**: The end points of the branches are the most basic actions or techniques an attacker would use.
  - **AND/OR Logic**: The tree uses logic gates (AND/OR) to show if an attacker must perform multiple steps (AND) or can choose any one of several options (OR) to succeed at a particular level.
  - **Threat Modeling**: Attack trees are a powerful tool for identifying vulnerabilities, understanding how a system can be compromised, and developing security measures to mitigate risks.

Training Assignment

- An online store displays laptop computers for sale. The customer enters to browse the store items. He can register the purchase order with the required laptop computer code and quantity in a special form to prepare it and inform him that the order is ready to be received. ***Note that the data in the store is public and the orders are not.***

Please for this exercise, determine the following:

- ***Assets***
- ***Thread***
- ***Thread Actors***
- ***Data-Flow Diagram***
- ***Trust Boundary***
- ***Attacks Vectors***
- ***Attacks Surfaces***